Sufficiently Advanced Agents Are Indistinguishable from Erlang: An Operational Mapping from OTP to Multi-agent LLM Systems

Mick Darling✉
Dollhouse Research
Boston, USA
mick@dollhouseresearch.com

Abstract

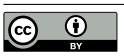
Multi-agent LLM systems are rediscovering the engineering problem that produced Erlang/OTP: how to build systems more reliable than their individually unreliable parts. We treat this convergence as a predictive engineering model rather than an analogy. We develop a term-by-term correspondence between OTP constructs and agent systems, grading twenty mappings as exact, loose, or breaking; the breaking mappings mark the real limits of the transfer: context isolation, payload trust, and mid-inference preemption. The correspondence draws on production experience: Erlang ingest fleets processing millions of tweets per hour, and our present agent runtime's independent reproduction of failure shapes OTP was built to survive: orphaned workers, token-burning polling loops, and sessions held open across hour-scale delays. We then test supervision as a capability multiplier in preregistered experiments that cross supervision architecture with model tier. Across 2,400 episodes and eight model tiers, OTP-disciplined supervision never loses to bare agent loops, leads at four of the five tiers where the arms separate, and shows its largest edge where models partially succeed (pass^4 at 3B: 95% supervised vs. 65% bare; at 32B: 70% vs. 45% with retry and 5% without). A preregistered matched-compute control and a prospective replication (640 episodes in total) rule against the evaluated best-of-N control as the explanation: supervision beat nominally token-matched best-of-N in both (paired deltas +0.33 and +0.26, $p \leq 0.004$), and the preregistered selector added zero successes over its first candidate; compute parity itself was not certified in either run. On hard decomposable tasks, supervised clean-context decomposition lifts a 32B model from 57% to 82%, monotonically from one to sixteen LLM actors, on a prospectively frozen holdout. We close with the problems our work to date leaves open: semantic failure detection, context leakage, and restart economics.

CCS Concepts

• **Software and its engineering → Software fault tolerance**; • **Computing methodologies → Distributed artificial intelligence**.

Keywords

multi-agent systems, LLM agents, Erlang/OTP, supervision trees, fault tolerance, automated software engineering



This work is licensed under a Creative Commons Attribution 4.0 International License.
MAS-GAIN '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2990-4/2026/10
https://doi.org/10.1145/3843778.3856135

ACM Reference Format:
Mick Darling. 2026. Sufficiently Advanced Agents Are Indistinguishable from Erlang: An Operational Mapping from OTP to Multi-agent LLM Systems. In *Proceedings of the 2nd International Workshop on Multi-Agent Systems using Generative Artificial INtelligence for Automated Software Engineering (MAS-GAIN '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3843778.3856135

1 Remembering Erlang

This work started with a waiting problem in DollhouseMCP, a multi-agent runtime we operate in production. Agents needed to wait on outside events, sometimes for hours, and an agent that waits from inside an LLM session pays tokens the whole time; one unattended polling component once consumed most of a weekly quota doing nothing. The solution was to make the MCP server the hub. Work moves from the LLM to the server, and the agent parks there at zero token cost until an outside event, a timer, or a server-side calculation releases it back to the LLM for its next steps. The same routing gives the runtime its security checkpoint, because every action passes through the server before the LLM can use a tool. Many agents can run this loop at the same time, even from different LLMs, one agent's events releasing other waiting agents.

The resulting architecture of using many small loops that monitor each other, recover when one fails, and start new ones as needed is not new, and we had used it before. Roughly fifteen years earlier we ran Erlang fleets against Twitter's garden hose stream at peaks in the millions of tweets per hour, starting an actor per piece of work and absorbing failures dynamically. Recognizing that architecture led us to Erlang/OTP (Open Telecom Platform), the language and library telecom infrastructure has run on since the 1990s [3]. The constraint set is the same, unreliable components inside systems expected to stay up, so the correspondence applies well beyond our runtime. We mapped OTP's constructs against the runtime in dated engineering issues in April 2026, applied the patterns deliberately, and ran the planned experiments in July.

This paper tests the OTP correspondence as a prediction. The claim is not that multi-agent LLM systems resemble Erlang. The claim is that they face the constraint set that produced OTP, and that OTP's constructs, discipline, and failure catalogs therefore transfer testably to agent systems, including the developer-facing agent infrastructure of automated software engineering. A prediction has to be allowed to fail, so we grade the prediction in three ways.

First, we develop a term-by-term operational mapping between twenty OTP constructs and agent behavior and processes, grading each correspondence exact, loose, or breaking (§2). The grade measures the analogy itself, not any one implementation, and the distribution is 12 exact, 5 loose, and 3 breaking. The three breaking

Table 1: All twenty graded correspondences between OTP constructs and agent-system analogs. The grade measures the analogy, not any one implementation: 12 exact, 5 loose, 3 breaking.

OTP construct	Agent-system analog	Grade
Lightweight process, free when idle	Agent parked awaiting events; no LLM session, zero tokens	exact
Registered name, `:global`	Queryable agent identity with a global registrar	exact
`:global.send`	Direct delivery to a named waiting agent	exact
Mailbox, selective receive	Mailbox store with typed message filters	exact
Message immutability, atomic receive	Append-only entries, content-hashed, atomic consume	exact
Supervision strategies (one-for-one, one-for-all)	Supervisor agents restarting workers with clean context	exact
Restart intensity limits	Restart ceiling with escalation	exact
Behaviours (`gen_server`, `supervisor`)	Agent roles as enforced interfaces	exact
DynamicSupervisor	Spawning workers from a template on demand	exact
ETS shared read state	Shared read-optimized state store	exact
`:pg` process groups	Group membership with multi-agent fan-out delivery	exact
`exit/2`, suspend, resume	Out-of-band kill, pause, and resume of agents at activation boundaries	exact
Per-process heap	Isolated per-session runtime state	loose
Hot code loading, `code_change`	Instruction swap on a parked agent; in-flight state migration is harder	loose
Let it crash	Kill a derailed activation and restart clean; no crash signal for a wrong answer	loose
Generational garbage collection	Context-window compaction; illustrative only	loose
Mnesia, distributed transactions	Compensation in the Saga style, not distributed ACID	loose
Share-nothing as substrate default	Agents leak through accumulated context; isolation is policy	breaking
Typed term payloads, matched rather than interpreted	Prose payloads are interpreted and can carry injection	breaking
Preemptive scheduling	No mid-inference preemption in current serving stacks; monitors and timeouts instead	breaking

rows mark genuine limits of the transfer, and they set the research agenda. The grades are backed by production evidence recorded in dated engineering issues: wherever our own runtime lacked an exact construct, the failure that OTP predicts appeared, and the standard OTP fix was the fix.

Second, we test the transfer where it should matter most, namely as a capability multiplier for small models (§3, §4). In preregistered experiments across 2,400 episodes and eight model tiers, OTP-disciplined supervision never loses to bare agent loops, leads at four of the five tiers where the arms separate, and shows its largest edge where models partially succeed. A matched-compute control and a prospective replication (640 episodes) rule against the evaluated best-of-N control as the explanation: in both runs, the preregistered selector added zero successes over its first candidate. That result does not depend on the compute-parity band, which both runs missed. On hard decomposable tasks, supervised clean-context decomposition lifts a 32B model from 57% to 82% on a prospectively frozen holdout, and the lift is monotonic from one to sixteen LLM actors. A separately provisioned trial reproduced the curve.

Third, we state what the correspondence does not give us (§7): semantic failure detection, context isolation, and restart economics are places where LLM agents genuinely differ from Erlang processes. We treat them as the research agenda, not as footnotes.

One result repeats: supervision converts model weakness into recoverable failure; small models fail often but cheaply, and an OTP-disciplined architecture turns those failures into successes. On the pilot suite, starting from a bare 3B model at 65%, adding supervision reached 95% while scaling the bare loop up to 70B reached 35%, and §6 states what the evidence supports and what it does not.

2 The Mapping, Including Its Failures

The convergence claim requires that the mapping be operational rather than metaphorical. To show that, we grade every correspondence, including the failures. The grades measure the analogy itself, not any particular implementation: the question for each row is whether an OTP construct has a faithful counterpart in agent behavior and processes, in anybody's runtime. *Exact*: the construct has a direct agent-system analog with the same semantics. *Loose*: the concept transfers, but the semantics differ in a documented way. *Breaking*: the construct has no faithful agent-system analog, because the substrates genuinely differ. We graded twenty constructs, and the distribution is 12 exact, 5 loose, and 3 breaking. Table 1 lists all twenty. DollhouseMCP, a multi-agent runtime we operate in production, supplies the operational evidence behind the grades.

The grading procedure is the same for every row: state the OTP construct's claim-relevant operational contract, then compare its agent counterpart across lifecycle, state ownership, communication and ordering, failure signaling, recovery, and scheduling semantics. Exact requires that every semantic the claim uses is preserved; loose preserves the function with at least one documented material mismatch; breaking means the evaluated substrate offers no faithful counterpart. Production incidents and experiments support or challenge a grade but do not define it. The mapping is author-coded qualitative analysis: the constructs were selected in dated engineering issues before the experiments ran, and no independent audit has been performed. Where an agent-side isolation is enforced by runtime policy, the mechanism is loose, while the absent share-nothing substrate default is graded separately as breaking.

We investigate every row to see whether the construct can be used as designed, modified, or mitigated.

2.1 The Exact Rows

Twelve constructs have direct agent-system analogs, and one of them provides the main economic benefit of the transfer. An Erlang process parked in a selective receive is nearly free; that is what famously made two million idle connections on a single node achievable. We use parked as a defined state: a parked agent retains an addressable identity and a server-side mailbox but holds no resident or outstanding LLM activation, consumes no inference tokens, so the provider can be entirely offline with no effect, and becomes eligible for a new activation only when an external event, a timer, or a server-side result is delivered. It is the same engineering property applied to a new resource: tokens instead of CPU. The token-burning poller in §1 failed because this construct was missing, and the fix in our runtime was to build it.

The strongest evidence that these rows are exact comes from our own gaps. Wherever our runtime lacked one of the constructs, the failure that OTP predicts appeared in production, and the fix

that worked was the standard OTP fix. Our mailbox entries were editable after append and read-validate-consume was not atomic; the mapping predicted a tamper window there before we saw one, and the fix is a transcription of Erlang's semantics: append-only mailboxes with content hashing and atomic consume. Our supervisors had no restart ceiling, which OTP predicts becomes a runaway loop, and supervisors that did not tie worker lifecycles to client sessions left orphaned workers running for days after those sessions had died. Our single-waiter delivery could not express fan-out, the need OTP answers with `:pg` process groups, and our mailboxes served read-heavy state badly, which is why OTP ships ETS. Each fix was designed in a dated engineering issue in April 2026, before the matching incidents appeared in production. A construct whose absence produces the predicted failure, and whose standard fix works, is a construct that maps.

2.2 The Loose Rows

Five concepts transfer with documented differences. Per-session isolation lacks OTP's heap guarantee. Hot code loading works as an instruction swap on a parked agent, but migrating in-flight state across the swap is an open protocol problem. Let it crash transfers cleanly for state faults, because a derailed activation can be killed and restarted with a clean context, but two of its assumptions do not hold: an agent that is wrong emits a well-formed, confident message instead of crashing, and a restart costs tokens and seconds instead of microseconds. Context compaction behaves like generational garbage collection, graded loose because the comparison is illustrative and no design decision depends on it. And distributed transactions do not transfer as constructed; the agent-system answer is compensation in the Saga style [5], not distributed ACID. The costs on the let-it-crash row, semantic failure detection and restart economics, reappear as research problems in §7.

2.3 The Breaking Rows

Three constructs have no faithful agent-system analog. For the first two rows the difference is the substrate, not the engineering.

Erlang gives every process its own heap, and influence between processes flows only through messages; the agent substrate defaults the other way. Within one agent everything observed accumulates in the context, and between agents some architectures append every output to a shared history [13, 17], so isolation is a policy that must be built and policed rather than a guarantee. Followed to its end, that default is the failure: every received message appends to the same session until the window is exhausted, and everything the session has seen contaminates everything after it. Our runtime prevents this by construction. No actor is addressed by appending to a live session; every activation is a fresh context assembled by the supervising layer, and anything an actor needs from earlier work is passed in explicitly. That cost is measured: the supervised arm's margin over the retry arm, which keeps its accumulated context, appears in §4; the margin belongs to the integrated configuration, of which context isolation is one component. Deliberate sharing also works through a supervised external store, the pattern of Erlang's own escape hatch, ETS; our exploratory workspace results reach exactly that pattern (§6). What stays breaking is the guarantee: an Erlang process cannot reach another's state, while our isolation is an audited policy that a prose payload could subvert.

Erlang message payloads are structured terms that a receiving process pattern-matches against expected shapes rather than interpreting as instructions; agent payloads are prose that the receiving model interprets, so a message can carry an injection attack. We are developing a typed semantic instruction layer intended to narrow this gap; it is preliminary and unevaluated, and we claim nothing for it here beyond marking the row an open problem under active work.

And, Erlang preempts any process at reduction boundaries, while nothing in the serving stacks available to us can preempt a model mid-inference, so a stuck call blocks its agent and supervisors must rely on monitors and timeouts instead; third-party evidence that step-boundary delivery suffices for mid-course correction bounds the cost of this break (§5). We grade this third row as the least absolute of the three. Generation does pause between token steps, and an operator who controls the serving stack could in principle suspend and resume work at those boundaries. Hosted inference APIs expose no such control, we know of no deployed agent architecture that provides it, and whether it could be made practical is a cost and engineering question that current architectures leave open. The grade records the present state, not a law of the substrate. These three rows mark the real limits of the transfer, and they define the research agenda in §7.

Some contemporary frameworks adopt actor constructs consciously; that is an engineering choice. Our claim is about convergence: runtimes that did not set out to rebuild OTP keep rebuilding OTP, badly, one incident at a time, because the constraint set forces it. The observer effect weakens this evidence: our own designers knew Erlang, so our runtime's trajectory is recall as much as rediscovery. The convergence evidence that generalizes comes from the field as a whole: the failure taxonomy derived independently by others matches OTP's catalog (§5). The mapping's predicted-fix record, with fix designs dated before the incidents, is the evidence that the correspondence is predictive rather than decorative. The convergence claim and the experimental claim rest on different evidence: recurring adoption supports piecemeal rediscovery, the graded mapping evaluates operational correspondence, production incidents test prediction, and the experiments that follow evaluate one integrated configuration against specified controls. Experimental uplift alone does not imply convergence or structural equivalence.

3 A Preregistered Apparatus

The experiments run on a standalone harness that instantiates the mapping's constructs directly: rooted supervision trees, one-for-one restarts with sliding-window escalation ceilings, stateless workers with a clean context per activation, bounded SQLite-backed mailboxes with typed envelopes, and let-it-crash recovery. The envelopes are simultaneously the raw data and the audit trail: every episode appends to a uniform telemetry schema from which every reported number can be regenerated.

Arms. A: the OTP-disciplined actor fleet. B: a bare agent loop with fixed retry, where retries keep the poisoned context. C: a bare loop with no retry. S: best-of-N sampling of the bare loop, token-matched to arm A. Arm S is the falsification control the

central claim depends on. Its preregistered selector is deterministic and zero-token: candidates parse against the grader's own output contract, a canonical majority vote decides where the budget funds multiple candidates, the earliest valid candidate wins otherwise, and the selector never sees gold or graders. Simple retry baselines are strong and cheap [10], and a supervision claim that has not beaten both retry and matched-compute sampling has only been tested against weak baselines.

Two kinds of actor. The word actor covers two different things in this system, and we keep them distinct. Programmatic actors are deterministic code: supervisors, routers, validators, graders, and the mailbox machinery. LLM actors are workers whose single activation is one clean-context model call. Every actor count N in this paper counts LLM actors only, and the programmatic layer stays fixed while N varies. Actors are scheduled sequentially throughout: no concurrency, latency, or throughput property is claimed or measured anywhere in this paper.

The supervisor contract. The supervisor observes programmatic lifecycle signals: process failure, timeout, an invalid envelope or an output-contract failure reported by a validator, and restart intensity within a sliding window. Its actions are to terminate an activation, start a fresh-context replacement, route or quarantine an envelope, and escalate once the restart ceiling is reached. Domain-semantic errors that pass the configured validators raise no signal and are outside this contract; that blind spot is the first open problem in §7.

Model ladder. Each episode pins exact provider and version strings for eight tiers: Qwen3-1.7B, Ministral-3B, Qwen3-8B, Qwen3-14B, Qwen3-32B, Llama-3.3-70B (hosted open-weight endpoints), plus Claude Haiku 4.5 and Claude Sonnet 5 as frontier references.

AI use. Anthropic Claude and OpenAI Codex assisted with research synthesis, development and review of software and analysis code, data analysis, and manuscript preparation. All reported quantities were regenerated from sealed evidence using deterministic scripts; the author reviewed the generated work, independently verified the citations and results, and assumes responsibility for the complete manuscript.

Tasks and metrics. The tasks are multi-step tasks in the GAIA style [12] with normalize-then-exact-match grading, at three difficulty levels, and the scaling studies use harder decomposable semantic families: multi-hop proof graphs, provenance resolution, fault-hypothesis graphs, and constrained allocation. The harder families are proxies for core software-engineering work, because fault-hypothesis graphs stand in for debugging and provenance resolution stands in for dependency triage; they are not an end-to-end software-engineering benchmark. The apparatus supplies the decomposition and routing plan, so the experiments test execution under supervised decomposition rather than autonomous task decomposition. We propose the supervision layer as the mechanism that makes agent fleets doing that work reliable, and registered grids on software-engineering task suites are the next confirmatory step. The headline consistency metric is pass^4, which requires that all four attempts succeed and follows the pass^k discipline introduced by τ-bench [18]. Statistical units follow the frozen designs: pass^4 is one binary task-level outcome; M2/M2b pair task means and bootstrap whole tasks with attempts kept together; actor scaling pairs task-attempt clusters. Preregistered exact Wilcoxon signed-rank tests and cluster-bootstrap intervals are reported [14].

Confirmatory procedure. Every confirmatory stage follows the same sequence. A dated preregistration freezes the grid, selection rule, statistics, and compute-parity band before any inference runs. Runs execute from immutable container images pinned by digest, after a keyless cloud gate verifies the full pipeline end to end at zero model cost. Task selectors run outcome-blind on baseline rows and are sealed before any scaling results are examined. Every run emits a checksummed evidence manifest, and results are append-only with history never rewritten. Verdicts that miss their own preregistered gates are recorded as misses and never upgraded after the fact.

Because the paper's claims are about reliability engineering, we apply the same discipline to the evidence pipeline itself: two of the program's results in §4 and §6 are verdicts our own gates withheld, reported alongside the wins because they show the gates reject results as designed.

The supervision machinery itself is validated deterministically before any paid experiment relies on it: an injected-fault suite covering worker crashes, timeouts, duplicate arrivals, out-of-order streaming, and mid-task disappearance completed 18 of 18 expected outcomes with exact replay and zero model calls.

Evidence manifests, preregistration digests, and the replay tool are available to reviewers on request; they are not yet publicly archived. They permit independent audit of the reported analysis and byte-identical regeneration of every table at zero API cost, an audit property rather than experimental reproduction. Re-running the experiments requires the harness, model endpoints, tasks, and treatment configurations, so exact independent reproduction is claimed only for components actually released. Our intent is to release the measurement apparatus as open source under the AGPL, from the harness, scheduler, and envelope schemas through the validators, scoring, sealing, and audit code; the tested agent configurations are the treatment rather than the instrument, some tested here stay private, and every treatment runs behind the same interface and emits the same envelopes, provenance, and accounting.

4 Results

Three preregistered experiment families are frozen as of this submission: the pilot capability grid (M1), the matched-compute control and its replication (M2, M2b), and the actor-scaling holdout with an separately provisioned reproduction (MR1). All numbers below are from sealed, replayable evidence; none were selected after the fact.

4.1 Supervision Versus Bare Loops (M1)

M1 crossed three arms with all eight tiers: 25 task bundles with four attempts each, which comes to 2,400 episodes. Five failure-comparison bridge bundles per tier are excluded from capability scoring, so each cell reports pass^4 over 20 capability tasks (Table 2). The task, not the episode, is the independent sampling unit; the four attempts define pass^4 for that task.

Three tiers are uninformative, both frontier tiers at 100% and the 1.7B tier at 0% for every arm: the architecture cannot show uplift where every arm is perfect or none ever succeeds. Of the five

Table 2: M1: pass[4] by tier and arm (A supervised, B retry, C bare); 2,400 episodes. Each cell counts capability tasks, out of 20, with all four attempts succeeding (80 episodes per cell). Bold marks the leading arm where arms differ.

Tier	Model	A (sup.)	B (retry)	C (bare)
T0	Qwen3-1.7B	0/20 (0%)	0/20 (0%)	0/20 (0%)
T1	Ministral-3B	**19/20 (95%)**	13/20 (65%)	13/20 (65%)
T2	Qwen3-8B	**1/20 (5%)**	0/20 (0%)	0/20 (0%)
T3	Qwen3-14B	**1/20 (5%)**	0/20 (0%)	**1/20 (5%)**
T4	Qwen3-32B	**14/20 (70%)**	9/20 (45%)	1/20 (5%)
T5	Llama-3.3-70B	**14/20 (70%)**	7/20 (35%)	7/20 (35%)
F1	Claude Haiku 4.5	20/20 (100%)	20/20 (100%)	20/20 (100%)
F2	Claude Sonnet 5	20/20 (100%)	20/20 (100%)	20/20 (100%)

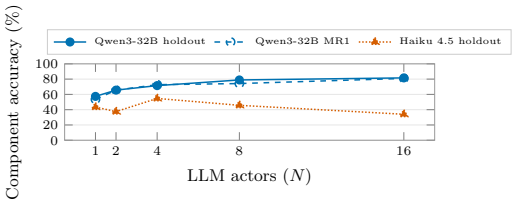


Figure 1: Clean-context decomposition on hard decomposable tasks: the Qwen3-32B holdout curve rises monotonically and MR1 reproduces it; Haiku 4.5 peaks at N=4 and declines beyond it.

remaining rungs, the supervised arm leads both controls at four (3B, 8B, 32B, 70B) and ties the stronger control at 14B, where arm A and the bare arm both sit at 5% while the retry arm sits at 0%. At 3B, the supervised arm scores 95% against 65% for both controls. At 32B, the supervised arm scores 70% against 45% with retry and 5% without. At 70B, the supervised arm scores 70% against 35% for both controls. The pattern is that supervision's edge is largest where the model partially succeeds: recovery needs failures to act on and successes to reach, so the edge vanishes at the floor and at the ceiling. The ladder is also capability-heterogeneous across model families on this suite (the 3B rung outscores 8B and 14B), so every claim is paired within a tier, never across rungs.

We report both controls everywhere deliberately. Retry alone recovers a large share of failures (45% versus 5% at 32B), so a supervision paper that compares only against a no-retry loop uses an artificially weak baseline. The supervised arm's remaining margin over retry, +25 points at 32B, is the gain associated with the integrated OTP-disciplined configuration. A retry that keeps the poisoned context repeats the original failure, and removing that context is one mechanism the design is intended to provide, but this experiment does not ablate clean context from the configuration's other supervision and messaging components.

We are explicit about what this grid can and cannot support. Twenty tasks per cell is pilot resolution: a single cell carries a wide binomial interval, and the 1/20 rungs are individually indistinguishable from noise. We therefore claim no per-cell significance. The grid's evidence is the pattern across rungs, read exactly as preregistered, and the statistical weight of this paper's claims rests on the paired comparisons and the frozen holdout that follow, where the confidence intervals are computed and shown. The higher-resolution grid is the registered next step, and every existing result regenerates from its sealed evidence.

4.2 The Evaluated Best-of-N Control Does Not Reproduce the Advantage (M2, M2b)

The strongest rival explanation for M1 is that supervision is just spending its restarts on more samples. M2 tested that directly: a preregistered comparison of supervised arm A against best-of-N sampling of the bare loop at matched token budget, at the 3B, 32B, and 70B tiers over 480 episodes; M2b then replicated the 32B comparison prospectively on held-out, never-scored attempts over 160 episodes. Table 3 reports both runs: supervision beat best-of-N in each, and the paired deltas agree.

One number matters more than the deltas: across all 320 best-of-N episodes in the two runs, the preregistered deterministic selector added exactly zero successes over its first candidate. This rules against the evaluated selector and schedule as the explanation for the observed advantage. It does not establish that sampling, voting, judging, or ensemble policies in general lack value, and the pattern is consistent with clean recovery rather than wider sampling.

One caveat remains: compute parity. Both runs carried a preregistered band requiring that realized best-of-N token spend fall within 10% of arm A's, and both missed it in opposite directions (Table 3). Both verdicts are therefore recorded as directional rather than certified. The misses taught us that compute parity cannot be scheduled statistically and must be enforced by a governor that never sees outcomes.

4.3 Decomposition Under Supervision

The scaling question is whether supervision can improve results rather than only recover from failures: does splitting a hard task across N clean-context supervised LLM actors beat the same model attempting it whole? An earlier sealed phase on easier task families produced large gains for reasons other than reasoning: a 3B model overloaded by a sixteen-item batch rose from 41.8% at one actor to 76.6% at four and Sonnet rose from 0% to 100%, but both gains came from removing batch-size and output-contract bottlenecks, and the 32B model, with no bottleneck on these families, sat at 99.2% and gained nothing (Table 6). Decomposition there repaired mechanical limits rather than demonstrating semantic uplift, so an outcome-blind selector correctly halted the phase before touching the frozen holdout. The harder calibration families, which cover multi-hop proof graphs, provenance resolution, fault-hypothesis graphs, and constrained allocation, left measurable headroom, and the prospectively frozen holdout of 480 rows, completed 2026-07-14, produced the program's cleanest result (Table 4, Figure 1).

Qwen3-32B climbs monotonically through every actor count from 57.4% at one actor to 81.6% at sixteen: +24.2 points, 95% CI [+13.3, +35.5]. Every point except N=2 returned full component coverage, and N=2 covered 248 of 256, so the gain is genuine semantic uplift rather than format repair. The intermediate point matters for practice: N=4 already delivers +14.1 points at roughly 12% fewer mean tokens than N=1 on the holdout's own telemetry.

A separately provisioned matched-resource trial (MR1, 240 episodes, run from a separately gated immutable image) reproduced the unconstrained curve, monotonically at every actor count: 53.5%

Table 3: M2 and M2b: supervised arm A versus token-matched best-of-N sampling of the bare loop (S). Selection gain counts successes added over the first sample. Verdicts are directional: neither run landed inside its compute-parity band; S/A is the realized 32B token ratio.

Run	Tiers	Ep./arm	Arm A	Best-of-N	Delta	95% CI	Wilcoxon p	S/A tokens (32B)	Sel. gain	Verdict
M2	3B, 32B, 70B	240	198/240	118/240	+0.33	[+0.225, +0.450]	$\approx 1.9 \times 10^{-6}$	0.836	0	directional
M2b (replication)	32B	80	60/80	39/80	+0.26	[+0.0875, +0.425]	0.0035	1.117	0	directional

Table 4: Frozen-holdout component accuracy versus LLM-actor count at all five prospectively pinned counts.

LLM actors	Qwen3-32B	Haiku 4.5
1	57.4%	43.4%
2	65.6%	37.5%
4	71.5%	54.7%
8	78.9%	45.7%
16	81.6%	34.0%

at one actor to 80.9% at sixteen, +27.3 points, cluster-bootstrap 95% CI [+16.4, +38.7]. The same trial's matched fixed-token and fixed-cost comparisons were withheld by their own preregistered validity gate: 23 of 160 governed episodes fell outside resource tolerance, so no matched deltas exist to report. The replication is therefore evidence for the curve itself, and explicitly not evidence about its cost.

Decomposition has costs and does not help every model. Haiku 4.5 shows a model-specific optimal granularity on the same holdout rather than monotonic scaling: a noise-level dip at N=2 (37.5%), a peak at N=4 (43.4% to 54.7%), a decline to 45.7% at N=8, and real harm at N=16 (34.0%). And we exclude Sonnet 5's numbers from every claim in this paper: its apparent gains on this suite are a contract phase transition, a strict-JSON batching failure at low LLM-actor counts that decomposition incidentally repairs, not reasoning uplift. LLM-actor count behaves like a per-model, per-task control variable. Two capability signatures have been observed (monotonic, and peaked at a model-specific optimal granularity) plus one measurement artifact (the contract phase transition), and a supervisor must anticipate all of them; §6 depends on that observation. Two curves cover little of this space: how capability changes with LLM-actor count across model sizes, families, and kinds of models is an open experimental program, and the apparatus is built to run that sweep tier by tier, preregistered and replayable, as it qualifies each rung.

5 Related Work

Actors and OTP. The actor model begins with Hewitt [8]. Erlang/OTP is its industrial descendant, and Armstrong's thesis [3] is the canonical statement of the problem itself: how to build reliable systems from unreliable components. This paper's claim is that the problem has returned. Some agent frameworks adopt actor constructs consciously; AutoGen's event-driven redesign is explicit about its actor lineage [17], and the adoption is now complete in frameworks that build agents directly on actor runtimes: Jido runs agents under OTP supervision trees on the BEAM itself [1], and Akka's agentic platform orchestrates agents with actor supervision on the JVM [2]. That depth of adoption also shows where the transfer breaks: Jido supervises LLM actors and deterministic actors with the same machinery, watching for the crash a process emits rather than the confident wrong answer a model produces, so the three breaking rows in §2 stay broken even there. Conscious adoption is an engineering choice and we make no novelty claim about it. Our subject is the runtimes that did not choose it and are converging on it anyway, driven by repeated production incidents.

Failure taxonomies. MAST [4] established from 1,600+ annotated traces that most multi-agent failures stem from system design and orchestration rather than model capability. MAS-FIRE [9] showed by fault injection that supervised, closed-loop topologies neutralize a large fraction of faults that collapse linear pipelines. Both are consistent with our results, but neither runs the comparison that this paper runs: preregistered supervision arms against matched-compute controls across a model-capability ladder. Our mapping also predicts the overlap: if agent runtimes are convergently rebuilding OTP, the field's independently derived failure taxonomy should match OTP's failure catalog, and it largely does.

Selection without verification. Our zero-uplift best-of-N finding has parallels at frontier scale: published scaling attempts that sample widely and select without a verifier repeatedly lose their gains at the selection step [6]. We measured the same failure at small-model scale, where the evaluated selection step contributed exactly zero across 320 best-of-N episodes. Independent measurement is also arriving on the mailbox rows: AgentRadio [15] isolated a gain of over ten points from adding passive step-boundary delivery of peer messages, `gen_server`-style delivery without preemption, to unmodified coding agents, in an ablation published eight days after this paper was submitted.

Runtime-architecture patterns for agents. The closest academic prior we have found is Srinivasan [16], which composes runtime patterns for production LLM agents by drawing on supervision and saga lineages, and which argues that a supervisor must sit between LLM output and the world. That work selects patterns; ours grades the full correspondence between OTP and agent systems, backs the grades with production evidence, and measures the effect of the supervision discipline across a capability ladder. A practitioner essay by Guimarães [7] states, without measurements, a thesis adjacent to ours: that agent frameworks are re-deriving BEAM-style infrastructure. We cite it as an independent observation of the same trend, which is itself weak evidence for the convergence claim.

Evaluation discipline. Our metric and statistics choices follow the reliability-evaluation line: pass^4 consistency from τ-bench [18], GAIA-style verifiable tasks [12], cost-aware comparison against strong simple baselines [10], and clustered uncertainty on the task unit [14]. The long-horizon measurement line [11] has no published agentic time-horizon for any 32B-class open model, the gap the small-model case of §6 addresses.

Table 5: OTP pattern coverage in this program. *Tested*: sealed, preregistered evidence. *Spot*: deterministic validation or sealed exploratory runs. *Gap*: designed against our runtime, no experimental result yet; every gap row is an open research question.

OTP pattern	Status	Evidence or state
Supervision tree, one-for-one restart, escalation ceiling	tested	M1, M2/M2b, holdout, MR1 (3,760 episodes); leads or ties both controls at every rung; leads at four of five informative rungs
Stateless worker, clean context per activation	tested	the arm-A-versus-retry separation; +25 points over retry at 32B
Clean-context decomposition, dynamic fan-out	tested	holdout 57.4% to 81.6%, monotonic; separately provisioned reproduction
Bounded mailboxes, typed envelopes, audit-trail messaging	tested	substrate of every run; append-only, hash-checked, byte-identical replay
Fault injection and recovery semantics	spot	18/18 deterministic outcomes (crash, timeout, duplicate, disappearance, streaming); paid fault grid unrun
Supervised message topologies	spot	six replayable graphs, answer-preservation only
Externalized supervised workspace (ETS-adjacent)	spot	accuracy preserved at 30% fewer tokens (1.7B); uplift not established
Specialized behaviour roles (model-backed curator)	spot	1.7B curator, 16/16 dispositions against a deterministic control
Long-horizon supervision, durable episodes	spot	calibration only; registered confirmatory design unauthorized
Restart intensity policies	spot	implemented and exercised; never independently ablated
Enforced behaviour contracts	gap	roles are conventions today; a worker can illegally dispatch and nothing stops it
Hot code loading with migration	gap	works as a demo; documented envelope-versioning race; never benchmarked
Publish-subscribe fan-out; out-of-band control plane	gap	designed; single-waiter delivery and "hit escape" today
Cross-agent transactions (Saga semantics)	gap	designed; deliberately absent from the runtime
Lifecycle links at OS-process scale	gap	orphaned long-running workers are the missing primitive, observed
Concurrent scheduling; wall-clock properties	gap	everything to date is sequential by design
Heterogeneous fleets; synthesizing reducers	gap	the largest unexplored capability axes in the program
Control-loop supervision over continuous observables	gap	mechanism below; telemetry already logs the state vector

Table 6: The numbers behind Table 5's spot tier plus the measured negative results, all from sealed, replayable runs; none is a confirmatory claim.

Configuration and tier	Result	What the number counts
Injected-fault suite (deterministic)	18/18	expected self-test outcomes; the comparative fault grid is the next registered run
Supervised topologies	6	replayable message graphs, answer-preservation only
Externalized workspace, 1.7B / 32B	−30% / −32%	tokens, with baseline accuracy and full success retained
Model-backed curator, 1.7B	16/16	correct dispositions vs. a deterministic control; curated projection 28% smaller
Long-horizon fleet, 32B	16	steps, the full horizon, completed in every episode past step zero
Homogeneous scaling, easier families, 3B	+34.8 pp	component accuracy at N=4 vs one actor (41.8% to 76.6%, CI [+14.8, +55.1]); batch-bottleneck removal
Homogeneous scaling at a ceiling, 32B	−2.7 pp	at N=16 on the same easier families; a model at 99.2% gains nothing from more actors
Model-generated decomposition plans	0	parseable plans; the oracle-split control also lost to no-split
Proof-preservation configuration	0/8 to 6/8	Haiku lifted; Sonnet regressed; promotion gate failed
MR1 matched-resource comparisons	23/160	governed episodes outside resource tolerance; verdict withheld

6 The Capability, Measured and Predicted

The results in §4 are point measurements; Table 5 maps the larger capability they sample. The tested tier is narrow and deep, the spot tier varies fault, topology, workspace, and horizon, and every gap row has a dated but untested design against our runtime. Programmatic components handle supervision, routing, validation, deduplication, escalation, replay, and budget enforcement; the model performs only the residual semantic work. The unit of capability is the configured system, not the model inside it. This split enables deterministic guarantees around a stochastic core and predicts that the measured curves will extend as tasks are decomposed into manageable components.

Measured, exploratory tier. Table 6 collects labeled calibration evidence, from workspace curation that preserved accuracy at roughly a third fewer tokens to a sixteen-step horizon completed in every episode past step zero. These are sealed, replayable existence results, not confirmed claims; they extend the decomposition to state curation, workspace hygiene, and horizon extension.

Measured, negative tier. Model-generated decomposition plans failed completely in one calibration, and the oracle-split control also lost to the no-split baseline, so decomposition quality is currently a programmatic responsibility; the failures we measured were protocol failures under one prompt, not evidence against the capability, so where that boundary sits remains open. A proof-preservation configuration lifted Haiku while regressing Sonnet and failed its promotion gate, and the matched-resource economics of scaling remain open because the first governed trial was invalidated by its own audit (Table 6). We report negative and withheld results because they mark measured limits of the capability.

Predicted, near-checkable. Three predictions follow from that account and have registered or buildable tests. First, injected-fault grids should widen the supervision gap, because arm B's defining weakness is inherited poisoned context, and injected faults produce exactly that condition; the paid grid is the next confirmatory run. Second, a parity tier should exist: some supervised small-model configuration matching an unsupervised frontier baseline on a defined suite. M1 is suggestive, supervised 3B exceeded bare 70B (95% versus 35%), though that is an unregistered cross-tier comparison; the certified version, with its cost ratio, is left to a registered publication-grade run. Third, the granularity signatures of §4 should be predictable from model and task features; once they are, LLM-actor count stops being a hyperparameter and becomes a control variable.

That third prediction leads to what we believe supervision becomes next; the mechanism follows from the row where the OTP mapping breaks most severely. OTP supervision is bang-bang control: the supervisor observes one bit (alive or crashed) and has one action (restart). LLM agents instead fail continuously, through drift, degradation, and token burn, none of it raising a crash signal. The natural extension is classical feedback control layered on the supervision tree. The state is a per-agent vector, progress, tool-error rate, retry rate, token burn, and drift, sampled per tick against the agent's declared setpoint; the actuator is typed, validated, reversible configuration change that activates and deactivates declared capabilities, never prompt injection. Controllers stay small and fast,

plain code or small models, never a frontier model in the loop. The derivative term is the one OTP lacks, escalation on degradation rate before failure arrives: an orphaned worker can drift for days without ever crashing, but a D-term supervisor would plausibly act within minutes. Similarly, the curated-workspace results suggest that the reasoning substrate itself can move into an externalized, supervised workspace where every write is validated, versioned, and auditable: working memory becomes as auditable as mailbox envelopes already are.

Our expectation is that reliability engineering becomes a requirement at every model tier, not a small-model workaround; at the small tier it is the dominant capability multiplier we observe. Current practice buys reliability by buying model scale; our data points the other way: a supervised 32B fleet finished above the Haiku 4.5 frontier reference's best supervised cell on the hard holdout (81.6% versus 54.7%), and the parity-tier prediction is the registered form. Nothing in the mechanism is scale-specific: larger models fail through hallucination and misalignment without programmatic structure around them, and the same machinery applies. We expect these tools to make small fleets rival mid-frontier models and frontier models both stronger and safer; different models demonstrably respond differently, so per-model configuration is an open problem.

7 What This Work Leaves Open

Three differences between LLM agents and Erlang processes remain the research agenda.

Semantic failure detection. Erlang's entire discipline depends on one unambiguous signal: processes crash. A hallucinating agent emits well-formed, confident, wrong messages, and nothing crosses the supervisor contract of §3; every validator we deploy is a partial, per-domain reconstruction of the crash signal. The control-loop extension in §6 is our proposed direction because it treats degradation as a continuous observable; semantic failure detection remains the largest open hole in the transfer.

Context leakage and isolation. Erlang enforces process isolation in the runtime; we enforce agent isolation as policy over a system that accumulates context by default. Policy can be audited but also subverted. Mailbox trust tiers, contract-checked payloads, and the typed semantic instruction layer of §2 are designed but not proven.

Restart economics. Erlang restarts cost microseconds; ours cost tokens, seconds, and money. Clean restarts recover capability (§4), but agent restart strategy must be cost-aware, and the cost figures remain uncertified (§6).

We use OTP as a default and depart where it fits badly: message passing is a bad fit for long-running computation, speculative execution, read-heavy shared state, and tight inner loops, and Erlang's own answer was named escape hatches, not doctrine; ours is the same.

Conclusion. An orphaned worker drifting for days shows what happens without a supervision tree; a 32B model climbing from 57% to 82% under clean-context decomposition shows what happens with one. Between them is a graded, falsifiable correspondence of twelve exact, five loose, and three breaking rows, plus a preregistered apparatus for continued testing. The breaking rows show where the analogy fails in the systems and tools we evaluated, not where it must always fail. We have concrete plans to narrow several of these gaps, although those plans are not evaluated here. Erlang processes cannot read or change one another's private state; agent isolation is enforced by policy. Erlang code matches structured messages; LLMs interpret prose, including instructions that can redirect them. Erlang can interrupt running processes; hosted model APIs cannot pause and resume generation. Instruments are available to reviewers on request, with open-source release intended (§3).

Data Availability Statement

Aggregate data for Tables 2–4 and Figure 1, mapping evidence matrices, provenance notes, and the generation script are supplied as auxiliary material. Full evidence and harnesses contain proprietary, patent-relevant implementation material and are available on request under appropriate confidentiality.

References

[1] Agent Jido Project. 2026. Jido: Autonomous Agent Framework for Elixir. Open-source software, version 2.0.0. https://github.com/agentjido/jido

[2] Akka. 2025. Announcing the Akka Agentic Platform. Blog post. Retrieved September 12, 2026 from https://akka.io/blog/announcing-akkas-agentic-ai-release

[3] Joe Armstrong. 2003. *Making Reliable Distributed Systems in the Presence of Software Errors.* Doctor of Technology dissertation. Royal Institute of Technology (KTH), Stockholm, Sweden. https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-3658

[4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail?. In *NeurIPS 2025, Datasets and Benchmarks Track.* doi:10.52202/085713-4082

[5] Edward Y. Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. doi:10.48550/arXiv.2503.11951

[6] Ryan Ehrlich, Bradley Brown, Jordan Juravsky, et al. 2025. CodeMonkeys: Scaling Test-Time Compute for Software Engineering. doi:10.48550/arXiv.2501.14723

[7] George Guimarães. 2026. Your Agent Framework Is Just a Bad Clone of Elixir: Concurrency Lessons from Telecom to AI. Blog post. Retrieved September 12, 2026 from https://georgeguimaraes.com/your-agent-orchestrator-is-just-a-bad-clone-of-elixir/

[8] Carl Hewitt, Peter Bishop, and Richard Steiger. 1973. A Universal Modular ACTOR Formalism for Artificial Intelligence. In *Proceedings of IJCAI '73.* 235–245. https://www.ijcai.org/Proceedings/73/Papers/027B.pdf

[9] Jin Jia, Zhiling Deng, Zhuangbin Chen, et al. 2026. MAS-FIRE: Fault Injection and Reliability Evaluation for LLM-Based Multi-Agent Systems. doi:10.48550/arXiv.2602.19843

[10] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, et al. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). https://openreview.net/forum?id=Zy4uFzMviZ

[11] Thomas Kwa, Ben West, Joel Becker, et al. 2025. Measuring AI Ability to Complete Long Software Tasks. In *NeurIPS 2025.* doi:10.52202/085713-3086

[12] Grégoire Mialon, Clémentine Fourrier, Craig Swift, et al. 2024. GAIA: A Benchmark for General AI Assistants. In *ICLR 2024.* https://openreview.net/forum?id=fibxvahvs3

[13] Microsoft. 2026. Microsoft Agent Framework Workflows Orchestrations - Group Chat. Microsoft Learn documentation. Retrieved September 12, 2026 from https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/group-chat

[14] Evan Miller. 2024. Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations. doi:10.48550/arXiv.2411.00640

[15] Xinxing Ren, Qianbo Zang, Ziyan Wang, et al. 2026. AgentRadio: Passive Awareness for Long-Horizon Multi-Agent Collaboration. doi:10.48550/arXiv.2607.28430

[16] Vasundra Srinivasan. 2026. A Methodology for Selecting and Composing Runtime Architecture Patterns for Production LLM Agents. doi:10.48550/arXiv.2605.20173

[17] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *COLM 2024.* https://openreview.net/forum?id=BAakY1hNKS

[18] Shunyu Yao, Noah Shinn, Pedram Razavi, et al. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *ICLR 2025.* https://openreview.net/forum?id=roNSXZpUDN

Received 2026-07-22; accepted 2026-08-25